

06 — Code Map

Audience: anyone asking "where do I change X?" **Companion docs:** [01 Architecture](#) · [08 Register](#) · [14 Configuration](#)

Two ways in: browse the tree (§2–§10), or jump straight to the **"where do I change X?"** index (§11).

1. Top level

```
vox/  
├─ services/          10 runtime services + the React UI  
├─ packages/          2 shared Python libraries  
├─ deploy/            compose, Helm, nginx, the deploy script, UI image  
├─ docs/              specification, runbooks, ADRs, OpenAPI, and this handbook  
├─ postman/           API collections (CRUD, E2E journeys, demo users)  
├─ scripts/           generators and dev utilities  
├─ README.md QUICKSTART.md CONTRIBUTING.md BACKEND_STANDARDS.md CHANGELOG.md  
└─ Makefile
```

2. `packages/evam-backend-core` — the spine

Every FastAPI service imports this. **Cross-cutting security lives here and nowhere else.**

Module	Responsibility
<code>app.py</code>	FastAPI app factory — shared middleware, error handlers, docs
<code>config.py</code>	<code>BaseServiceSettings</code> ; each service subclasses with its own env prefix
<code>db/session.py</code> , <code>db/base.py</code>	Async engine, session lifecycle, declarative base
<code>crud.py</code>	Generic CRUD primitives
<code>pagination.py</code>	Keyset pagination
<code>errors.py</code>	Problem-envelope error shapes
<code>logging.py</code>	Structured logging, JSON in production
<code>middleware.py</code>	Request id, timing, correlation
<code>health.py</code>	<code>/healthz</code> , <code>/readyz</code>
<code>router.py</code>	Router conventions
<code>retry.py</code>	Transient-failure retry (<code>docs/adr/0006-transient-retry.md</code>)
<code>rbac_catalog.py</code>	Roles, tiers, aliases, the <code>Access</code> <code>IntEnum</code> , <code>POLICY_VERSION</code>
<code>rbac.py</code>	The compiled baseline: <code>OPERATIONS</code> (~70) + compatibility re-exports
<code>lifecycle.py</code>	<code>STAGE_VOCAB</code> , <code>INITIAL_STATUS</code> , <code>ALLOWED_TRANSITIONS</code> , <code>ROW_LOCKS</code>
<code>policy.py</code>	The scope + stage/field policy engine: <code>MANDATORY_FOR_STAGE</code> , <code>FIELD_LOCKS</code> , <code>EVIDENCE_FOR_STAGE</code> , <code>check_write()</code>
<code>internal_token.py</code>	Mint/verify the signed internal context (HS256 / RS256)
<code>service_policy.py</code>	<code>SERVICE_GRANTS</code> — what each machine identity may do
<code>oidc.py</code>	Token verification, multi-issuer
<code>evidence.py</code>	Governance evidence helpers

The bolded modules are the security invariants. Changing one is a reviewed decision, and usually needs the UI mirror updated too.

3. `packages/evam-register-client`

Typed HTTP client for the Register, used by ATLAS, PULSE, VocX and the workflow activities. **Nothing should hand-roll Register calls.**

4. `services/register` — the book

```

app/
├─ api/                34 routers
│  ├─ crud_router.py   ← the factory every table's surface comes from
│  ├─ resources.py     ← the ResourceSpec registry (16 CRUD resources)
│  ├─ rbac.py          assignments · requests · approve · lead convert · authz check
│  ├─ reconciliation.py the import quarantine queue
│  ├─ custom.py        audit · dossier · documents upload · /v1/ref
│  ├─ imports.py export.py export_ledger.py
│  ├─ lms.py tranches.py sanction.py covenants.py ews.py cpcs.py
│  ├─ handover.py advaya.py decisions.py notifications.py notify.py
│  ├─ documents_lifecycle.py evidence.py calendar.py followups.py
│  ├─ series.py closure.py people_sync.py tenants.py
│  └─ entity_rules.py people_rules.py ← pre-write / pre-delete hooks
├─ authz/
│  ├─ engine.py        applies RBAC + policy to a request
│  ├─ matrix.py        the effective matrix from the signed context
│  ├─ scope.py         row scoping for SCOPED users
│  └─ revalidate.py    online revalidation against Access
├─ core/
│  ├─ security.py      RequestContext – how identity enters the service
│  ├─ config.py enums.py errors.py logging.py middleware.py
│  ├─ access_client.py people.py reconciliation.py pagination.py router.py retry.py
│  └─ evidence.py
├─ db/                session.py · base.py · apply_qls.py ← RLS lives here
├─ models/            20 modules → ~43 tables (see §11)
├─ repositories/      crud.py · documents.py · financials.py · interactions.py · subjects.py
├─ schemas/           base.py · rbac.py · resources.py
├─ seed/              refdata.py · loader.py · from_xlsx.py · ledger_xlsx.py · templates/
└─ storage/           base.py · s3.py (MinIO)

```

Model modules → domains

Module	Tables
registry.py	entities, people, counterparties, ref_values
deals.py	leads, deals
trackers.py	lending_tracker, syndication_tracker, syndication_lenders, asset_monetisation, financials, contracts_assets, external_intelligence, monitoring_reporting
interactions.py	interactions
documents.py	documents, document_checklist
lms.py	loan_accounts, loan_ledger_entries, loan_account_conditions, disbursement_tranches
covenants.py	covenants
ews.py*	ews_cases
sanction.py	sanction_terms, cam_reports, cam_turns
cpcs.py	cp_cs_checklists
advaya.py	advaya_handoffs, advaya_handover_packages
decisions.py	workflow_decisions, workflow_decision_outbox
notifications.py	notifications, notification_deliveries
calendar.py	calendar_events
evidence.py	governance_evidence, governance_evidence_status
reconciliation.py	import_reconciliation_items
series.py	number_series
users.py	line_assignments, change_requests
system.py	idempotency_keys, tenants, tenant_settings
prism.py	cross-cutting mixins

* `ews_cases` is declared alongside the covenant/monitoring models.

5. `services/gateway` — the one door

```

app/
├─ main.py          verification · header stripping · minting · proxying · slow paths
├─ routes_map.py    (method, path regex) → RBAC operation ← the edge gate
├─ resolver.py      Access lookups + permission cache
└─ config.py        upstream URLs and per-service injected keys

```

Four things live here and nowhere else: `_SKIP_REQUEST_HEADERS` (the forgery defence), `_SLOW_PATHS` (the long-timeout allowlist), `_route()` (prefix → upstream), and `ROUTE_OPERATIONS`.

6. `services/access` — identity

```

app/
├─ api.py           users · roles · matrix · resolve · drift · version · me
├─ models.py        tenants · users · user_roles · access_grants · matrix_versions · access_audit
├─ matrix.py        effective-matrix computation
├─ seed.py          baseline seed + `--check` drift report
└─ security.py      schemas.py config.py main.py

```

7. `services/workflows` — the durable plane

```

app/
├─ workflows.py     14 workflows + the shared _Foundation SLA/run-control state machine
├─ activities.py    every Register call a workflow makes (~40 activities)
├─ api.py           the ORCHESTRATOR – HTTP front for start / signal / query
├─ worker.py        worker entrypoint (registers workflows + activities)
├─ codec.py         AES-256-GCM payload encryption for Temporal history
├─ notifier.py      outbox drain
├─ cam.py           the CAM workbench engine
├─ reconciler.py    drift reconciliation
├─ docx_out.py      document generation
├─ types.py         workflow input/result dataclasses
└─ config.py

```

Two processes from one image: `python -m app.worker` and `python -m app.api`.

8. `services/vocx` — voice capture

```

app/
├─ main.py  config.py
├─ vocx/
│   ├─ core/
│   │   ├─ server.py      the HTTP surface (capture, status, reports, drafts, auth)
│   │   ├─ pipeline.py    extract → resolve → gate → plan → execute
│   │   ├─ extract.py     transcript → structured fields
│   │   ├─ resolve.py     EntityResolver – match a spoken name to a register entity
│   │   ├─ gate.py        auto-write vs approval card; the write plan
│   │   └─ atlas.py store.py search.py
│   ├─ speech/
│   │   ├─ stt.py         Stub · FasterWhisper · API transcribers + build_transcriber
│   │   └─ audio_store.py the archive
│   ├─ google/           oauth.py · drive_writer.py · notes.py · workspace.py
│   ├─ registry/         store.py · writer.py
│   └─ identity.py loader.py mount.py reports.py

```

9. `services/stt`, `services/pulse`, `services/atlas`

Service	Key files
<code>stt</code>	<code>app/main.py</code> (OpenAI-compatible endpoint) · <code>app/engine.py</code> (faster-whisper, <code>cpu_threads</code>) · <code>app/config.py</code>
<code>pulse</code>	fetch → <code>matching.py</code> → signal rules → Register with an <code>Idempotency-Key</code>
<code>atlas</code>	stateless BFF: <code>/v1/dashboard</code> , <code>/v1/today</code> , <code>/v1/pipeline/{vertical}</code> , <code>/v1/entities/{id}/summary</code> , <code>POST /atlas/cache/invalidate</code>

10. `services/atlas/ui` — the React SPA

```

src/
├─ api/          axiosClient.ts · vocxClient.ts · http.ts · mockData/
├─ auth/         rbac.ts (the UI mirror) · session.ts · AuthContext.tsx
├─ components/
│   ├─ common/   Field · CommonTable helpers · ConfirmDialog · ExportBar · Pills · StatCard ·
│   SubTabs · InteractionRow · ErrorBoundary · PageHint
│   ├─ layout/   AppLayout · Navbar · BottomNav · navConfig.ts
│   ├─ table/    CommonTable.tsx
│   └─ vocx/     VocxProvider · VocxPanel · VocxLauncher · RecordTab · useRecorder.ts ·
│   ReportsTab · ApproveDialog · ClientPicker · LogToPicker
├─ workflow/     ActionsPanel · ActionFormDialog · CamWorkbenchDialog · CpcsChecklistDialog ·
│   DisburseDialog · HandoverPackageDialog · SanctionTermsDialog · ExecutedAgreementDialog
├─ copilot/
├─ pages/        Home · Today · Dashboard · Leads · Deals · Lending · Syndication ·
│   AssetMonetisation · Clients · FIMaster · Employees · Masters · Audit · Activity · Tools · Login
├─ services/     one module per domain (see below)
├─ context/
├─ utils/
└─ assets/

```

The `services/` layer is the seam

Every page talks to a service module; no page calls axios directly.

Module	Domain
<code>leadsService</code> <code>dealsService</code> <code>lendingService</code> <code>syndicationService</code> <code>assetMonService</code>	The four books
<code>clientsService</code> <code>entitiesService</code> <code>fiService</code> <code>employeesService</code>	Masters
<code>accessService</code> <code>authService</code>	Identity and roles
<code>nameResolver</code>	Joins normalised tracker rows to company name + code
<code>interactionService</code> <code>notesService</code> <code>documentsService</code>	Timeline and files
<code>workflowService</code> <code>workflowActionsService</code> <code>workflowRun</code> <code>stageRequestService</code> <code>conversionService</code>	The workflow plane
<code>vocxService</code> <code>camService</code>	Voice and CAM
<code>dashboardService</code> <code>activityService</code> <code>auditService</code> <code>auditDetail</code>	Read-side
<code>pulseService</code> <code>newsService</code> <code>notificationsService</code>	Intel and alerts
<code>lmsService</code> <code>ledgerService</code> <code>backupService</code> <code>referenceService</code>	Servicing, ledger, admin

`nameResolver.ts` **deserves a note.** The Register keeps grids normalised — a tracker row carries `deal_id` and `entity_id`, not a company name. This module fetches two small lookup maps (entity id → name+code, deal id → number+entity), caches them for 60 s, and joins client-side. It **fails soft**: a resolver outage leaves the joined columns blank, it never fails the grid that asked.

Building the UI

```
cd services/atlas/ui
VITE_USE_REAL_API=true npx vite build --outDir dist-live --emptyOutDir
```

Build-time variables: `VITE_API_BASE_URL` (default `/v1`), `VITE_ACCESS_URL`, `VITE_VOCX_URL`, `VITE_VOCX_MAX_SECONDS`. The container image (`deploy/ui-image/Dockerfile`) passes these as `ARG` / `ENV`.

There is **no automated test suite and no working eslint config** for the UI. Verification today is `tsc --noEmit` plus browser-driven checks. Treat that as a known gap when changing shared code such as `nameResolver.ts`.

11. Where do I change X?

I want to...	Go to
Add a role	<code>packages/evam-backend-core/evam_backend_core/rbac_catalog.py</code> and <code>services/atlas/ui/src/auth/rbac.ts</code>
Change what a role may do	Runtime: Access matrix via ATLAS. Baseline: <code>evam_backend_core/rbac.py::OPERATIONS</code>
Change which screens a role sees	<code>services/atlas/ui/src/auth/rbac.ts</code> (<code>VIEW_ROWS</code>) + the Access view matrix
Add a stage / change legal moves	<code>evam_backend_core/lifecycle.py</code> (+ <code>refdata.py</code> , cross-checked by a test)
Make a field mandatory at a stage	<code>evam_backend_core/policy.py::MANDATORY_FOR_STAGE</code>
Freeze a field after sanction	<code>policy.py::FIELD_LOCKS</code> / <code>lifecycle.py::ROW_LOCKS</code>
Add a dropdown value	<code>services/register/app/seed/refdata.py</code> → <code>/v1/ref</code> (no browser redeploy)
Add a table	<code>services/register/app/models/</code> + migration + <code>schemas/resources.py</code> + <code>api/resources.py</code>
Add a non-CRUD endpoint	a new module in <code>services/register/app/api/</code>
Gate a route at the edge	<code>services/gateway/app/routes_map.py</code>
Let a machine call something	<code>evam_backend_core/service_policy.py</code>
Add a service behind the gateway	<code>gateway/app/config.py</code> + <code>_route()</code> prefixes + compose/Helm
Add a durable business process	<code>services/workflows/app/workflows.py</code> + <code>activities.py</code> + <code>worker.py</code> + <code>api.py</code>
Change a timeout	<code>deploy/nginx/nginx.conf</code> and <code>gateway/app/main.py::_SLOW_PATHS</code> and the browser client and <code>gateway.ingress.slowPaths</code>
Change the recording length cap	<code>VITE_VOCX_MAX_SECONDS</code> (<code>deploy/ui-image/Dockerfile</code>); logic in <code>components/vocx/useRecorder.ts</code>
Change STT accuracy / speed	<code>services/stt/app/config.py</code> — model size, beam size, <code>cpu_threads</code>
Change the news rules	<code>services/pulse/app/matching.py</code> + <code>PULSE_RED_WORDS</code> / <code>PULSE_GREEN_WORDS</code>
Add a dashboard number	<code>services/atlas/app/</code> (BFF) + <code>pages/Dashboard/compute.ts</code>
Add a column to a grid	the page under <code>pages/</code> + its <code>*.types.ts</code> + the service module's row mapper
Fix a blank joined column	<code>services/atlas/ui/src/services/nameResolver.ts</code>
Change the import mapping	<code>services/register/app/seed/from_xlsx.py</code>

I want to...	Go to
Change backup/upgrade behaviour	<code>deploy/prism-deploy.sh</code>
Change TLS, headers, body size	<code>deploy/nginx/nginx.conf</code>
Change what production turns on	<code>deploy/compose/docker-compose.prod-posture.yml</code>

12. Tests and generated artefacts

Location	Contents
<code>services/*/tests/</code>	Per-service pytest suites — unit, integration, concurrency (no lost updates/deadlock)
<code>packages/evam-backend-core/tests/</code>	<code>test_internal_token.py</code> , <code>test_oidc_multi_issuer.py</code> , <code>test_policy.py</code>
<code>postman/</code>	<code>PRISM_All_APIs</code> , <code>PRISM_UI_CRUD</code> , <code>PRISM_E2E_Full</code> , <code>PRISM_E2E_Journey</code> , <code>Orchestrator</code> , <code>PRISM_Demo_Users</code> + environments
<code>docs/openapi/</code>	Exported specs for register, gateway, orchestrator
<code>scripts/</code>	<code>gen_all_apis.py</code> , <code>gen_postman.py</code> , <code>gen_e2e_*.py</code> , <code>gen_demo_users.py</code> , <code>gen_dev_certs.sh</code> , <code>install_edge_certs.sh</code> , <code>e2e_smoke.sh</code> , <code>export_openapi.sh</code> , <code>new_service.py</code> , <code>audit_*.py</code>

`scripts/new_service.py` scaffolds a new service against the shared core — use it rather than copying an existing service, so the conventions come along.

13. Conventions worth absorbing

Read `BACKEND_STANDARDS.md` and `CONTRIBUTING.md` in full once. The four that come up most:

1. **Never re-implement a cross-cutting rule.** Scope, lifecycle, token minting and machine grants each have exactly one implementation.
2. **A save path must never fail silently.** No `if (!found) return;` on a write.
3. **Reject unknown filters, do not ignore them.** Silently dropping a filter widens a result set the caller believed was narrow.
4. **Comment the *why*.** The existing code explains the reasoning behind a decision, not what the line does. Match that register.